

Prompt Optimization: How to Make Smaller Models Punch Above Their Weight

TL;DR

We explored using prompt optimization to make cheaper, smaller language models perform as well as expensive, more capable ones. We achieved up to 4x performance improvements for trace classification, while learning important lessons about overfitting along the way.

Key findings:

1. Prompt optimization can significantly improve model performance (10% → 50% accuracy on GPT 4o mini)
2. Optimized prompts transfer performance gains across different model families
3. Rich textual feedback in optimization can risk prompt overfitting
4. Synthetic datasets generated by an ensemble of top models can bootstrap optimization when no labeled data exists
5. Strategic validation set sizing and meta-prompting the teacher LM helps mitigate potential overfitting

With the increasing number of traces our users are logging using Orq, it can be arduous to maintain oversight by manually analyzing them. Hidden in these thousands of traces are valuable insights into what the deployment, agent or evaluator is actually being used for and what problems it might be running into. We have been working on a solution to effectively process and classify the traces, with the goal of delivering actionable insights to our users. One way to do this is by classifying the traces into topics and subtopics and identifying problems; a task well-suited for LLMs. However, due to the potentially very large number of traces, the cost can become a prohibitive factor. We recently wrote a [research blogpost](#) about fine-tuning LLMs to improve the performance of smaller language models on this exact task. Check that out if you are interested.

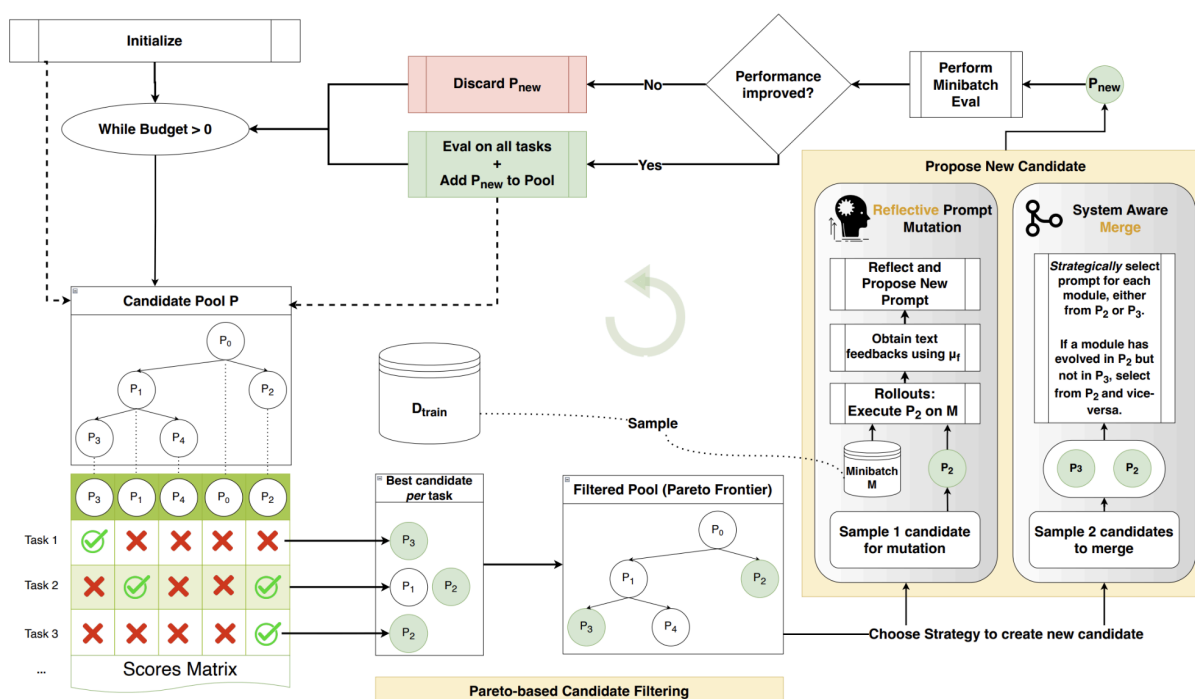
Another way to reduce costs is by choosing a smaller model and improving its prompt. We have noticed that larger, smarter models are seen to understand complex tasks more easily, without as much prompting needed. Smaller models need more prompting to arrive at the same level of performance, if they are ever able to. Therefore, a strategy can be to prompt a cheaper model such that it reaches the performance of smarter, more expensive ones. This process of iteratively improving prompts, known as prompt engineering, is something our platform is well-suited for. This can be done either manually or automatically through a process known as prompt optimization.

Prompt Optimization

There are many types of prompt optimization frameworks and approaches, here we chose to go with DSPy and their new GEPA algorithm. For more details regarding the framework check out

[Optimization Overview - DSPy](#) and [\[2507.19457\] GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning](#). In order to optimize a prompt you will need a dataset, ideally with expected outputs for each specific input. The dataset you use is crucial for the quality of the final prompt; the final performance will only ever be as good as the dataset used to optimize onto. When in a dataset drought, you can consider generating a dataset using one or multiple 'smart' LLMs, and then 'fitting' a smaller model onto this, achieving comparable performance at lower costs. We chose GEPA (Genetic Pareto) because it can achieve high sample efficiency – meaning it does not require a lot of data to work. This algorithm is also purely an instruction optimizer, as opposed to for example MIPROv2 that optimizes few shots as well as instructions, leading to longer overall prompts. GEPA specifically incorporates textual feedback along with the score, serving to further improve the quality of the next iteration of prompts.

Another critical component is the metric function, used to evaluate an output compared to the expected output. It must be set up manually and return a score and textual feedback. You start with an initial seed prompt and repeatedly use another 'teacher' LLM to rewrite the prompt, incorporating feedback from the metric. This is an evolutionary algorithm; we have a pool of candidates (prompts) and continually test new mutations of them, adding them to the pool if their performance improved. To avoid local optima, the selection for mutation uses a Pareto-based optimum: not simply choosing the best overall performing candidate but rather the best performing per datapoint.



Think of the algorithm as a group of players competing for multiple tasks. Each time the player that is the current best at one or multiple tasks is selected to either mutate or merge. Mutation

would mean trying out a sample of tasks, incorporating the feedback from the metric function and the teacher to learn from mistakes and change. Only if this new mutated player performs better at the sample of tasks, will they be added to the group. Merging strategically selects two players and, well, merges them, creating a single new player. Again this new player will only be added to the group if they perform better at a sample of tasks. In essence, it's an evolutionary approach with smarter selection and performance tracking.

Our Approach

There are multiple aspects to our trace insights, here we focused on the first part: trace topic assignment. Assigning a topic to a trace can help to build a taxonomy, creating a high-level oversight of the contents of the traces. Since we do not know all topics beforehand, the LLM is given the option to generate a new topic or assign an existing one. This must all be reflected in the dataset, and the metric function needs to score the outputs accordingly. For this task we had no data. Starting from zero meant that we needed to create representative input data points with corresponding outputs. It is recommended by DSPy to have at least 30 data points and ideally 300, so manual labeling is feasible. We used a collection of 5 top models (Opus 4.1, Sonnet 4.5, Gemini 2.5 pro, Kimi K2, GPT-5) to generate 5 datasets of outputs and then merged the answers, taking the most common ones as the ground truth. This sampling was done to create a strong dataset that minimizes individual model bias. The generated dataset of 45 points was then manually reviewed. Given the test/validation split, this left us at the lower end of the advised dataset size.

Our AI router provides built-in [support for DSPy](#). This makes it easy to switch between all the models you have enabled in Orq (see code). The traces that are made for each call also serve to provide clarification and control into what is being called under the hood. This was extremely useful for gaining visibility into what was happening and if everything was working as it should be. DSPy is a powerful framework, but is lacking in native observability. This made it easy to switch models while monitoring costs, precise contents of the calls and potential failures. Prompt optimization is an ever evolving field that has become more mature but still requires a lot of manual setup. We are working on bringing synthetic data generation and some form of prompt optimization to the platform, more on this soon.

```
```python
import os
import dspy

lm = dspy.LM('openai/openai/gpt-4o-mini',
 api_key=os.getenv("ORQ_API_KEY"),
 api_base="https://api.orq.ai/v2/proxy")
dspy.configure(lm=lm)
```
```

Optimization Results

First Optimization Run

The data below shows the scores of the baseline seed prompt versus an optimized prompt. In this example we ran the optimization on GPT 4o mini with Kimi K2 on Groq as the teacher LM. We chose Kimi on Groq because it was fast and intelligent; good bang for your buck. The baseline is a simple prompt explaining the task and its in and outputs. The scores are relatively low due to the difficult nature of the task, it must both assign existing topics and create new ones correctly. We scored a correct topic creation using semantic similarity (TF-IDF and cosine similarity) to the expected output. The final accuracy is the average of the assignment and creation scores. The results show a clear improvement of the accuracy on our generated dataset: almost 37% for GPT 4o mini, the model we optimized on. This optimization took approximately 10 minutes to complete. It ran for 450 metric calls, creating 48 iterations and set us back a mere \$0.48 for the entire run, hinting at the efficiency of the optimizer.

| Model | Baseline (%) | Optimized (%) | Delta |
|-----------------------|-------------------|-------------------|--------------|
| GPT 4o mini | 10.80 ±0.9 | 47.60 ±2.7 | +36.8 |
| GPT oss 20B | 9.0 ±3.7 | 49.20 ±3.7 | +40.2 |
| Gemini 2.5 flash-lite | 15.80 ±1.9 | 44.80 ±4.1 | +29.0 |
| Kimi K2 | 12.60 ±3.7 | 41.30 ±2.6 | +28.7 |
| Gemini 2.5 Pro | 12.60 ±1.86 | 49.70 ±0.76 | +37.1 |
| Claude Sonnet 4.5 | 13.80 ±1.8 | 50.30 ±4.7 | +36.5 |
| Average | 12.4% | 47.2% | +34.7 |

The accuracy of different models on the topic classification task, comparing a baseline versus the prompt that was optimized on GPT 4o mini.

One of our questions going into the research was: to what extent does an optimized prompt carry performance increases to other models on which the prompt was not optimized? So, we tested the baseline and the optimized prompt on other models and found an average increase of nearly 35%; the prompt that was optimized on GPT 4o mini carried its performance gains over to other models. The standard deviations of the results were relatively high, so the better performance of the prompt on models it was not even optimized on, e.g. Sonnet 4.5 reaching 50% accuracy or deltas higher than those of GPT 4o mini, are to be taken with a grain of salt. The results were promising, but taking a closer look we soon saw that this conclusion, a neat

performance transfer, was not entirely valid. The optimized prompt contained very specific instructions as for what topics to create or assign. In the excerpt below you can see an example of the type of rules that the teacher LM was adding to the prompt: an if-then statement containing the exact name of the topic. The performance increase, and its translation across models, is now certainly less spectacular; the prompt consisting of a collection of edge cases seen in the training data.

> 'When a user is confused by UI changes or cannot locate features, create a topic for "UI/UX Navigation & Product Feedback" even if "Technical Feature Configuration" exists, because usability and navigation are distinct from technical setup'

Prompt Overfitting

It can be argued that these few-shot examples translate to good performance on unseen data, i.e., good generalization. However, they can also work to overfit or bias the model on these examples, an effect that could be especially pronounced on smaller models. Due to the rich textual feedback that is given to the teacher LM, there is a higher risk of overfitting. Depending on how you set up your scoring function, the teacher LM has knowledge of the correct reference answer and will use this to improve the prompt. Naturally, one can achieve the best scores by learning the precise combination of inputs and outputs, but this hinders generalization. The validation test set is meant to counteract this by keeping that data out of the training loop, hiding the exact answers by only returning the score. For GEPA, it is recommended to "Maximize the training set size, while keeping the validation set just large enough to reflect the distribution of the downstream tasks (test set)."

Second Optimization Run

For the next run of optimizations we therefore decided to increase the size of the validation set from 20% to 30%. Additionally, we appended a sentence to the feedback that the teacher LM sees: "NEVER tell the assistant exactly what topics to create. Instead, set up rules that achieve the correct topics without specifying their names. Eg. broader, less broad, more focused on the learning objective..." This is an attempt to decrease the specificity of the new prompts and foster higher-level pattern creation. For the next run of optimizations we kept the same setup of optimizing on GPT 4o mini with Kimi K2 as a teacher. This time we compared the optimization against a handcrafted two-shot prompt as the baseline. The optimization ran for approximately 15 minutes using DSPy's auto=medium setting for deeper exploration. This meant 741 metric calls and 48 iterations, setting us back a total of \$0.98. The results are shown below.

| Model | Baseline (%) | Optimized (%) | Delta |
|------------------|--------------|---------------|-------|
| GPT 4o mini | 26.67 | 30.74 | +4.07 |
| Claude 4.5 Haiku | 37.20 | 38.76 | +1.56 |

| | | | |
|---------------------|-------|--------------|---------------|
| Qwen3 235b Instruct | 23.30 | 27.41 | +4.11 |
| GPT 5 mini | 9.07 | 22.78 | +13.71 |
| Gemini 2.5 flash | 8.80 | 38.70 | +29.90 |
| Gemini 3 pro | 23.40 | 35.74 | +12.34 |
| Average | 21.41 | 32.36 | +10.95 |

The accuracy of different models on the topic classification task, comparing a hand-crafted prompt as the baseline to the prompt optimized on GPT 4o mini.

The table shows an average increase in accuracy of about 11%, some having larger differences than others. The high variations in baseline performance are notable. This could be due to the way the data was generated, with there being two Claude family models in the pool. It can also allude to the fragility of and variance in performance of hand-crafted prompts across different models. The optimized prompt performs worse here than in the first run, indicating that the teacher LM took the instructions at heart and did not create equally specific if-then rules in the prompt. Looking closer, we can see that there was some guidance as to what to name and group topics into:

> When developers discuss API integration issues, environment differences, or rate limiting, these fall under 'Technical Support & Troubleshooting'

But it also gave a more guided decision framework to adhere to in the reasoning field that models produced before assigning or creating topics. Whether or not the overfitting was detrimental and the fix truly leads to better generalisability is uncertain for now. More data and research are needed to confirm it.

Conclusion

In this post, we explored how prompt optimization can be used to make smaller, cheaper language models perform comparably to larger, more expensive ones for trace classification tasks. We demonstrated that through Prompt Optimization using DSPy's GEPA algorithm, we could achieve significant performance improvements, boosting GPT-4o-mini from 10.8% to 47.6% accuracy. The optimized prompts successfully transferred their performance gains across different model families, suggesting transferability of optimized prompts.

However, we also discovered an important caveat: the rich textual feedback in the optimization loop, which is one of the things that makes GEPA powerful, can risk prompt overfitting, where the teacher LM encodes specific training examples directly into the prompt rather than learning generalizable patterns. By adjusting our approach, increasing the validation set size and

explicitly instructing the teacher LM to avoid naming specific topics—we aimed to capture higher-level reasoning patterns rather than memorized edge cases. More data is needed to conclusively support or disprove both the transferability and generalisability.

Taking into account that the dataset was generated you might be careful to not fit too precisely. If your dataset is larger and more representative of real world data, the tolerance for ‘overfitting’ can be higher; few-shot examples or rules like we saw above have been known to improve LLM performance. It all starts with the dataset you use—capturing edge cases and modelling behavior into a good dataset is foundational for successful optimization. Therefore, the prompt that is used to generate the dataset warrants thorough attention. Perhaps optimize that one as well...

Whether or not prompt optimization can work is very use case dependent. It is important to be able to set up a good metric function, and for open-ended tasks like summarization, one might have to rely on yet another LLM for evaluation. Orq's platform proved invaluable throughout the process of setting up and running the optimization, providing native DSPy integration with full observability into model calls, costs, and failures. This made it straightforward to experiment with different models while maintaining complete visibility into the optimization process. We're currently A/B testing the two optimized prompt versions on real-world traces to validate their performance in production.

****Key Takeaways:****

- Give special attention to a representative dataset and good metric function
- Small models can use prompt optimization to match larger models' performance using synthetic data
- Be mindful of overfitting when using rich textual feedback in optimization loops
- Validation set sizing and meta-instructions to the teacher LM can aid generalization

Next Steps

This work on trace topic classification is just one piece of our broader effort to help our users extract actionable insights from their traces. We're continuing to build out the full trace insights. Beyond trace classification specifically, we're also working on bringing more AI-powered capabilities directly to the Orq platform. This includes synthetic data generation to help users bootstrap their experiments and integrated prompt generation and eventually optimization. Our goal is to make it easy for both technical and non-technical users to leverage these advanced techniques to improve their AI lifecycles in Orq. Practical experience with latest techniques help us to integrate successful ones into our platform.

Stay tuned for more posts diving into other research as we continue to develop and test our solutions. Questions or want to discuss? Reach out to the Orq research team.
